



INT253

WEB DEVELOPMENT IN PYTHON USING DJANGO

Lecture #0

What is a Web Framework?

A web framework provides tools to create a website easily



Web



Frame



Work

What is Web Framework?



“ Web framework is a set of components designed to simplify your web development process. It has basic structuring tools in it, which serve as a solid base for your project. It allows you to focus on the most important details and project’s goals instead of creating things, that you can simply pull out of the framework. ”

What is Django ?



What is Django?

01

Django is a web application framework written in Python programming language.

02

It is based on MVT (Model View Template) design pattern.

03

The Django is very demanding due to its rapid development feature.

04

It takes less time to build application after collecting client requirement.

05

This framework uses a famous tag line: The web framework for perfectionists with deadlines.

History

/home/nitish-chauhan/Desktop/Django



**Publicly released
under BSD license.**

**2.0 version is
launched**

2003 — 2005 — 2008 — 2017 — 2018

**Django was design
and developed by
Lawrence journal
world.**

**1.0 version is
launched**

**Its current stable
version 2.0.3 is
launched.**

History of django

Inception (2003):

Django was developed by Adrian Holovaty and Simon Willison.

Started as an internal project at the Lawrence Journal-World newspaper in Kansas, USA.

Open Source Release (2005):

Released as an open-source web framework in July 2005.

Aimed at easing the development of complex, data-driven websites.

Django Software Foundation (2008):

Django gained community support and formed the Django Software Foundation (DSF) in 2008.

DSF oversees Django's development and promotes its use.

Version 1.0 (2008):

Django 1.0 was released in September 2008, marking stability and maturity.

Widely adopted in the web development community.

Introduction of Django REST Framework (2011):

Django REST Framework was introduced to simplify building Web APIs.

Enabled the development of RESTful APIs using Django.

Migration to Python 3 (2013-2019):

Django community worked on migrating the framework to Python 3.

Django 2.0, released in 2017, officially dropped support for Python 2.

Django Channels (2016):

Introduction of Django Channels for handling WebSockets and other asynchronous protocols.

Continued development and community support for Django.

Why use Django?

Why Django ?

1

Built on top of Python



Why Django ?

2

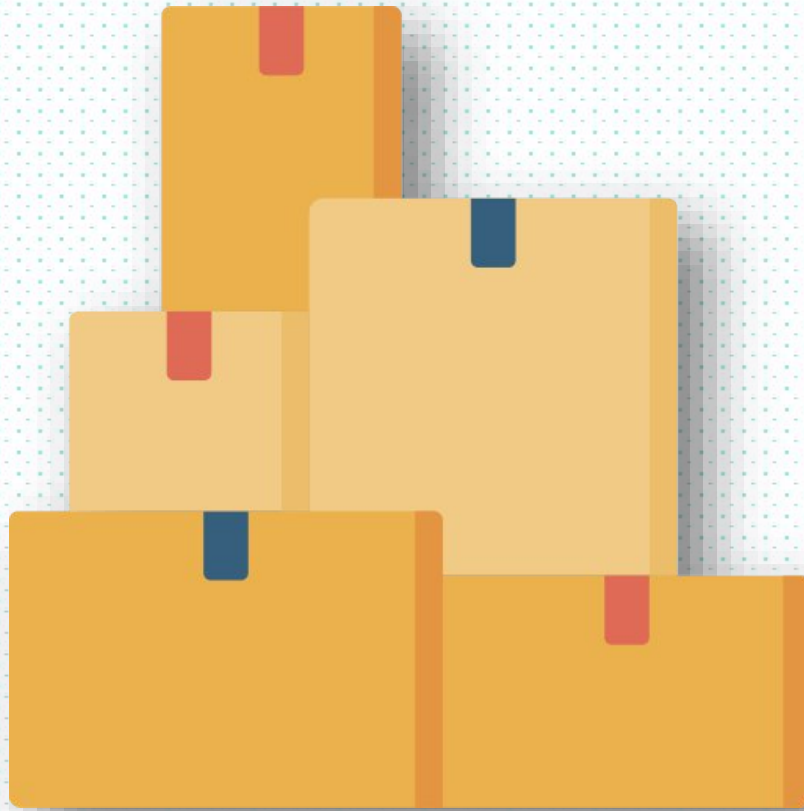
Quick development of websites



Why Django ?

3

Lots of packages



Why Django ?

4

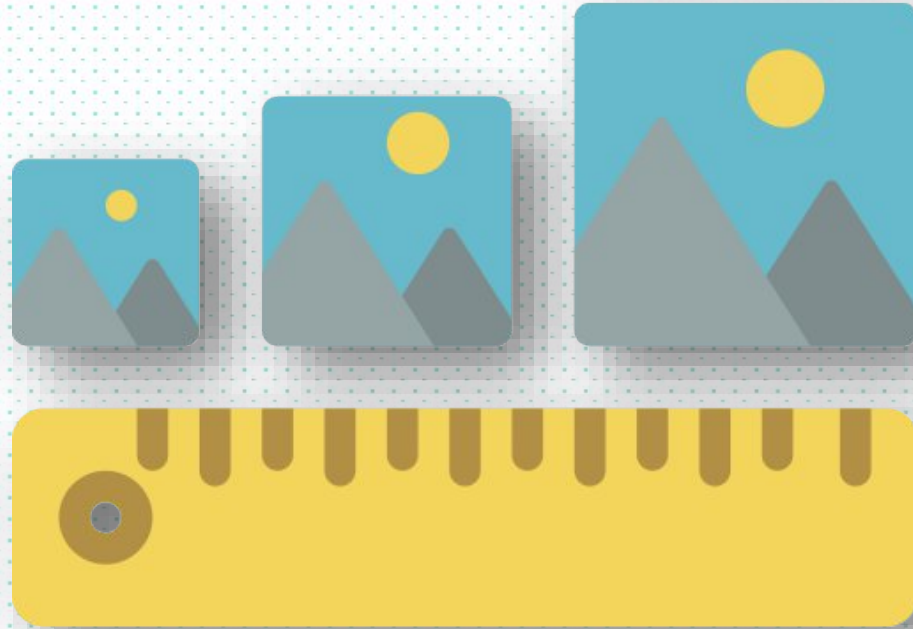
Django provides security



Why Django ?

5

Django is scalable



DJANGO APPS



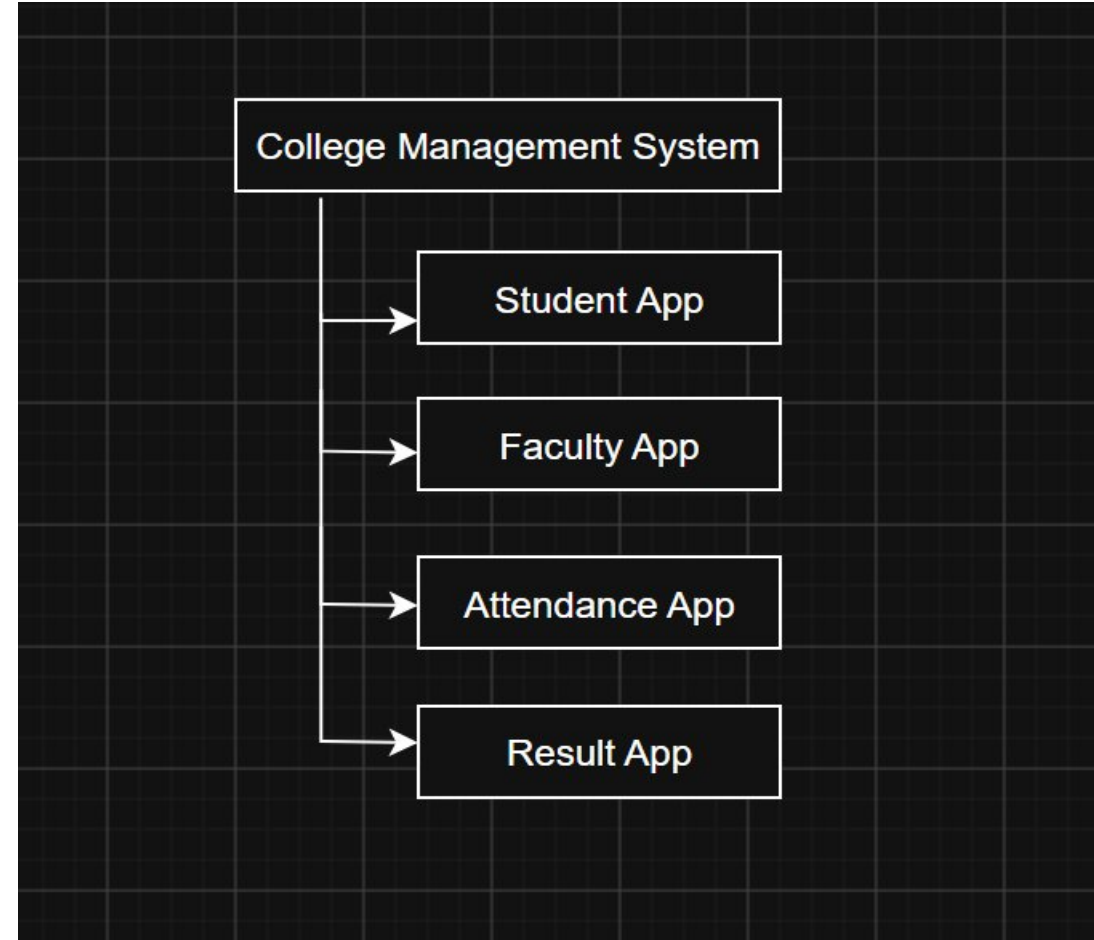
What is a Django Project?

- A Django Project is the top-level container of a Django application. It contains the global configuration, settings, URLs, and one or more Django applications (apps) that work together to build a complete web application.

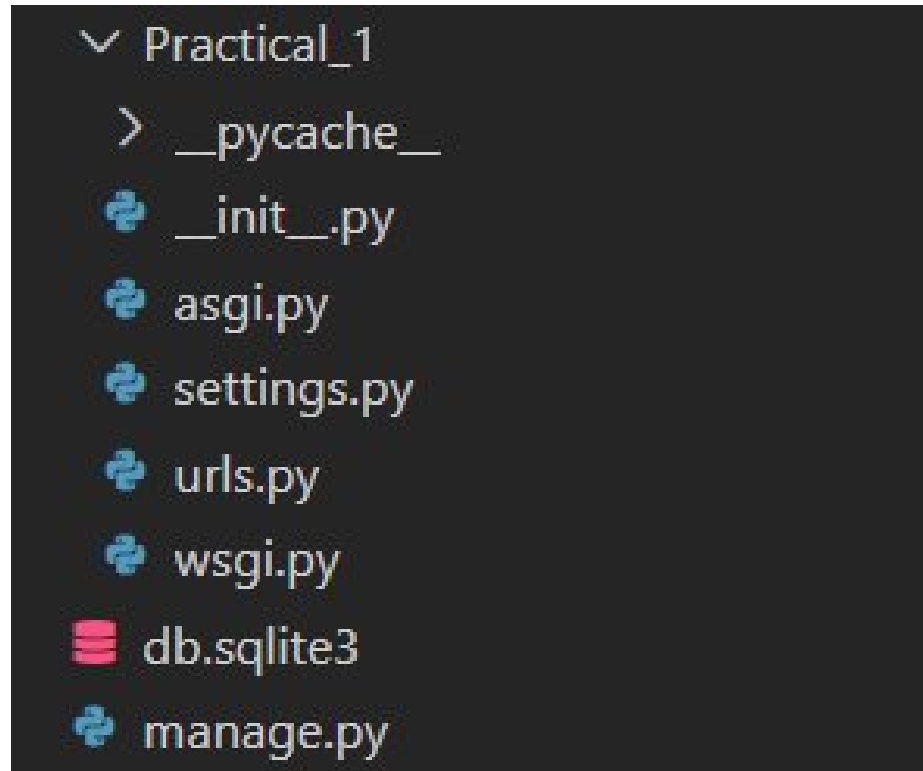
What is a Django App?

- A Django App is a self-contained Python module that implements a specific feature or functionality of a Django project. It contains its own models, views, templates, URLs, and other components required for that feature.

For Eg:

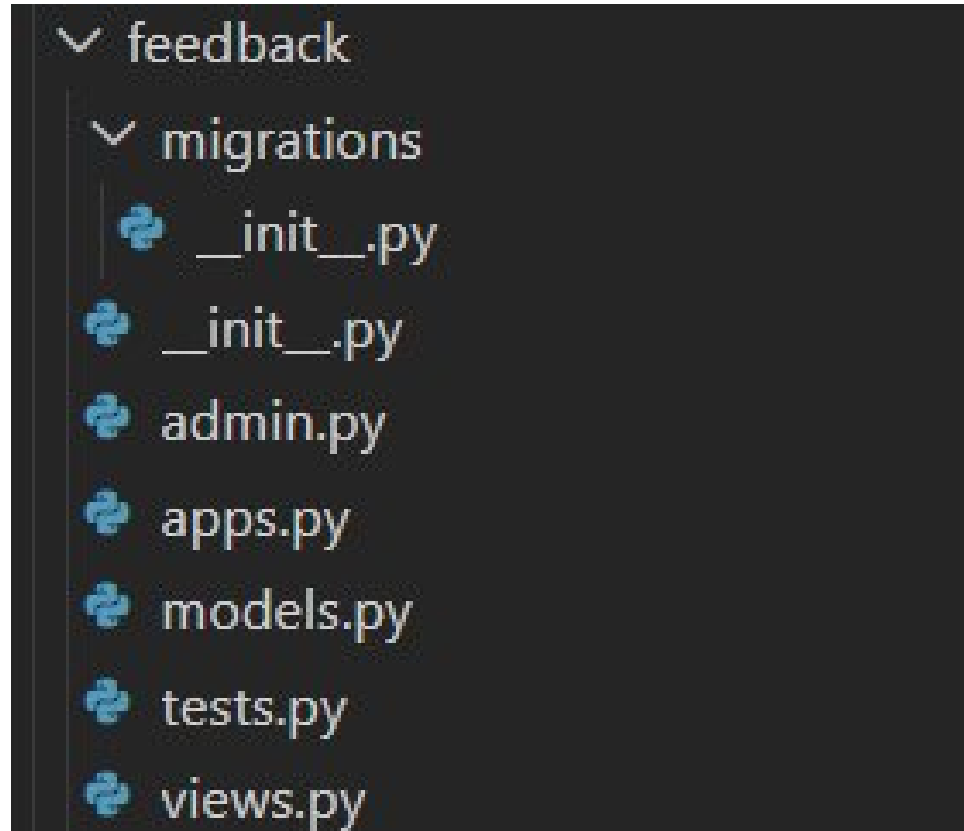


Folder structure – startproject command



- **manage.py** - manage.py is a command-line utility that allows developers to interact with the Django project.
- **init.py** - __init__.py tells Python that this folder should be treated as a Python package.
- **settings.py** - Contains all project-level configurations.
Everything about the project is controlled here.
- **urls.py** - Maps URLs to Views. - /student
- **wsgi.py** - (Web Server Gateway Interface)
- **asgi.py** - (Asynchronous Server Gateway Interface)

Folder structure – startapp command



- **init.py** - This folder (feedback) is a Python package.
- **apps.py** - It stores the configuration (basic information) of your app.
- **models.py** - This file is used to create database tables for storing data.
- **views.py** - This file contains the logic of your application.
- **admin.py** - This file connects your models to Django's Admin Panel.
- **tests.py** - This file is used to test whether your application works correctly.
- **migrations Folder** - A migration is a file that keeps track of changes made to your models.
- **migrations/init.py** - This file simply tells Python that the migrations folder is also a Python package.

django-admin and manage.py

django-admin

Used before creating a project.

Global Django command-line tool.

Creates new Django projects.

manage.py

Used after creating a project.

Project-specific command-line tool.

Manages the existing Django project.

Common django-admin Commands

1. Create a New Project

`django-admin startproject myproject`

Purpose: Creates a new Django project.

2. Check Django Version

`django-admin --version`

Purpose: Displays the installed Django version.

3. Display Help

`django-admin help`

Purpose: Shows all available Django commands.

Common manage.py Commands

1. Run Development Server

```
python manage.py runserver
```

Purpose: Starts the Django development server.

2. Create an App

```
python manage.py startapp feedback
```

Purpose: Creates a new Django app.

3. Create Migration Files

```
python manage.py makemigrations
```

Purpose: Detects changes in models and creates migration files.

4. Apply Migrations

```
python manage.py migrate
```

Purpose: Creates or updates database tables.

What is view?

View:

A View is a Python function that decides what to show to the user.

For Eg- whatever ui we are seeing

Creating View:

```
from django.http import HttpResponse  
def home(request):  
    return HttpResponse("Welcome to Django")
```

View with Logic and Decision:

```
def student(request):  
    if request.method == "POST":  
        return HttpResponse("Form Submitted")  
    return render(request, "student.html")
```

What is View?



A View is a Python function that decides what to show to the user.

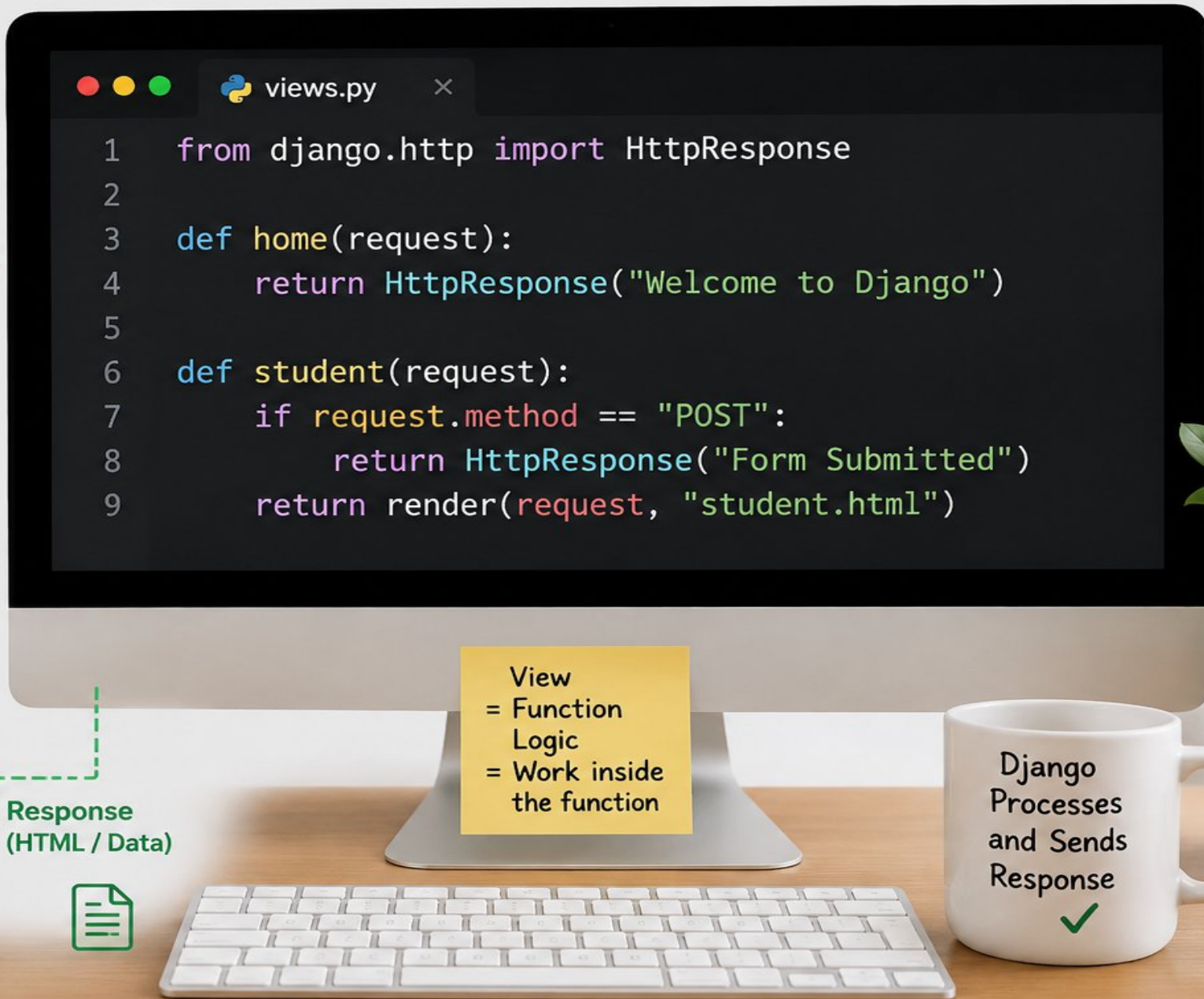


Creating View

```
from django.http import HttpResponse  
def home(request):  
    return HttpResponse("Welcome to Django")
```

View with Logic and Decision

```
def student(request):  
    if request.method == "POST":  
        return HttpResponse("Form Submitted")  
    return render(request, "student.html")
```



Response
(HTML / Data)



View logic

View logic means adding rules or decisions inside the view.

Example 1 (Without Logic)

```
from django.http import HttpResponse
```

```
def home(request):  
    return HttpResponse("Welcome to Django")
```

Every user sees:

Welcome to Django

No decision is taken.

Example 2 (With Logic)

```
def student(request):
```

```
    if request.method == "POST":  
        return HttpResponse("Form Submitted")
```

```
    return render(request, "student.html")
```

What happens?

If the user **opens the page**, show the form.

If the user **clicks Submit**, show **"Form Submitted"**.

The **if statement** is the logic.

Creating Views and View Logic

What is a View?



A View is a Python function that decides what to show to the user.

It contains the logic (rules and decisions) and returns a response.

Types of View Logic



1. Simple View

Returns a fixed response.



2. Conditional / Dynamic View

Uses conditions (if, else) and request data to give different responses.

1. Creating Simple View

```
from django.http import HttpResponse

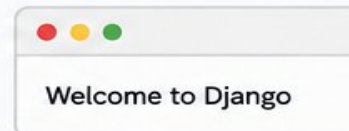
def home(request):
    # Simple logic
    return HttpResponse("Welcome to Django")
```

This view has no conditions. It always shows the same response.

Example:

When user opens
/home/

Output:



2. Creating View with Logic and Decision

```
from django.http import HttpResponse
from django.shortcuts import render

def student(request):
    if request.method == "POST":
        # Logic when form is submitted
        name = request.POST.get('name')
        return HttpResponse(f"Form Submitted. Hello {name}!")
    else:
        # Logic when page is first opened (GET request)
        return render(request, "student.html")
```

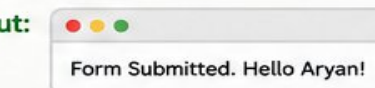
How it works:

- If user submits the form (POST), we get the data and show a message.
- If user just opens the page (GET), we show the HTML form (student.html).

Example:

When user submits form

Output:



How a View Works



1. User
opens a URL
in browser



2. URL is matched
in urls.py



3. Corresponding
view function is
called



4. View logic runs
(conditions, data
processing etc.)



5. View returns
HttpResponse /
render()



6. Response is sent
back and displayed
to the user

Key Points



- View = Python function
- It contains logic and decisions
- It returns a response (HttpResponse or render)
- Every page you see is controlled by a view

What is url mapping?

Url mapping:

URL Mapping means connecting a URL to a View.

For Eg- /about/ → about()

/contact/ → contact()

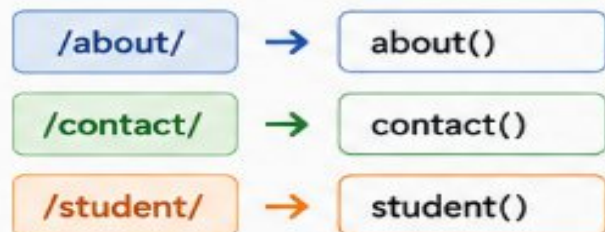
/student/ → student()

What is URL Mapping?

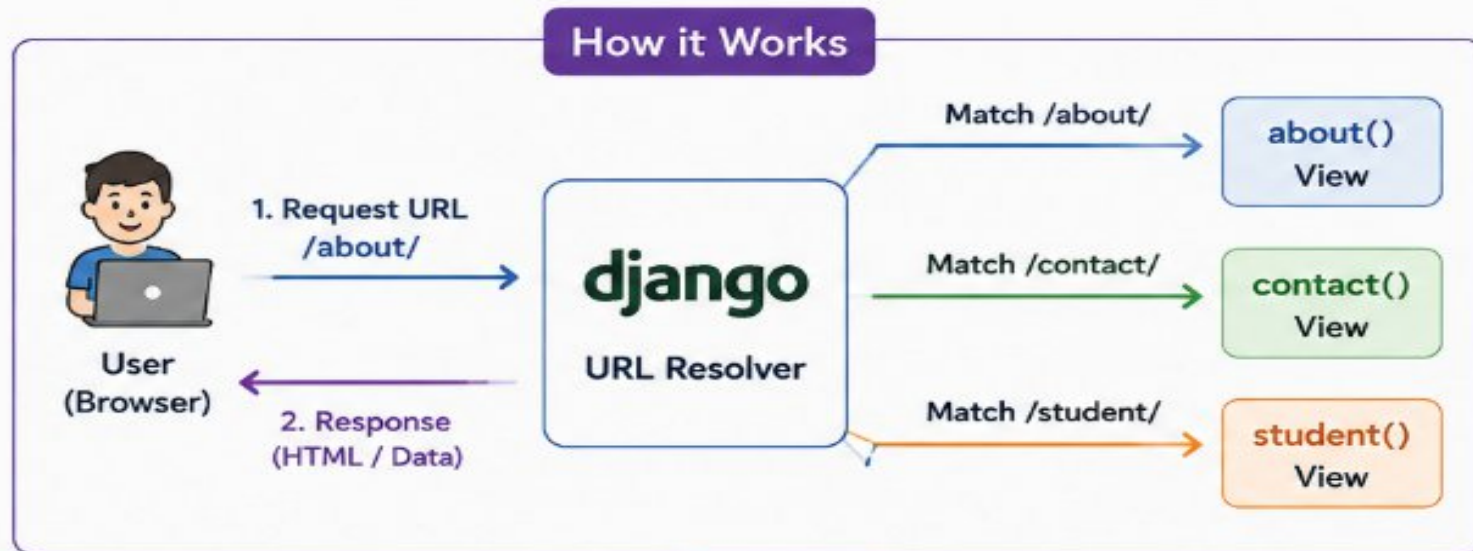
URL Mapping means connecting a **URL** to a **View**.

When a user enters a URL in the browser, Django matches the URL with a view function and shows the response.

Example Mappings:



How it Works



views.py

```
from django.http import HttpResponse

def about(request):
    return HttpResponse("This is About Page")

def contact(request):
    return HttpResponse("This is Contact Page")

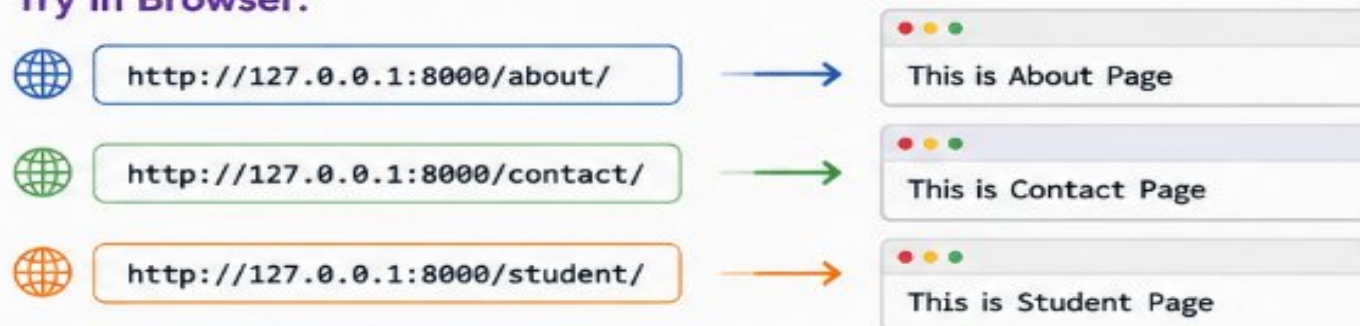
def student(request):
    return HttpResponse("This is Student Page")
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('about/', views.about),
    path('contact/', views.contact),
    path('student/', views.student),
]
```

Try in Browser:



In Simple Words:

- **URL** is what user types in browser
- **View** is the Python function that runs
- **URL Mapping** connects URL to the View

HTTP requests

- An **HTTP Request** is a message sent by the **browser (client)** to the **server** asking for something.

Browser asks → Server answers

Example : Opening a Website (GET Request)

You type:

`http://127.0.0.1:8000/about/`

Returns "About Us" page

Browser: "Please give me the About page."

Server: "Here it is."

Creating Views and View Logic

What is a View?



A View is a Python function that decides what to show to the user.

It contains the logic (rules and decisions) and returns a response.

Types of View Logic



1. Simple View

Returns a fixed response.



2. Conditional / Dynamic View

Uses conditions (if, else) and request data to give different responses.

1. Creating Simple View

```
from django.http import HttpResponse

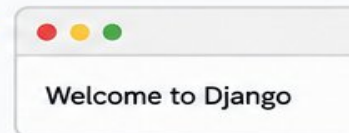
def home(request):
    # Simple logic
    return HttpResponse("Welcome to Django")
```

This view has no conditions. It always shows the same response.

Example:

When user opens
/home/

Output:



2. Creating View with Logic and Decision

```
from django.http import HttpResponse
from django.shortcuts import render

def student(request):
    if request.method == "POST":
        # Logic when form is submitted
        name = request.POST.get('name')
        return HttpResponse(f"Form Submitted. Hello {name}!")
    else:
        # Logic when page is first opened (GET request)
        return render(request, "student.html")
```

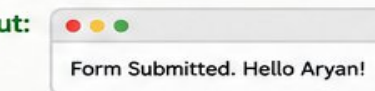
How it works:

- If user submits the form (POST), we get the data and show a message.
- If user just opens the page (GET), we show the HTML form (student.html).

Example:

When user submits form

Output:



How a View Works



1. User
opens a URL
in browser



2. URL is matched
in urls.py



3. Corresponding
view function is
called



4. View logic runs
(conditions, data
processing etc.)



5. View returns
HttpResponse /
render()



6. Response is sent
back and displayed
to the user



Key Points

- View = Python function
- It contains logic and decisions
- It returns a response (HttpResponse or render)
- Every page you see is controlled by a view

Creating Requests and Responses

Request:

A **Request** is an HTTP message sent by the client (browser) to the Django server asking for a web page, data, or to perform an action.

Response:

A **Response** is the HTTP message sent by the Django server back to the client containing the requested page, data, or result.

- **Real Life Example**
- Student: "Can I have a glass of water?"
- This is a **Request**
- Teacher: "Here is your water."
- This is the **Response**
- Exactly the same thing happens in Django.

Example

Form Example

Student enters

Name: Aryan

and clicks **Submit**.

Browser sends

Name = Aryan

to Django.

This is the **Request**.

Django replies

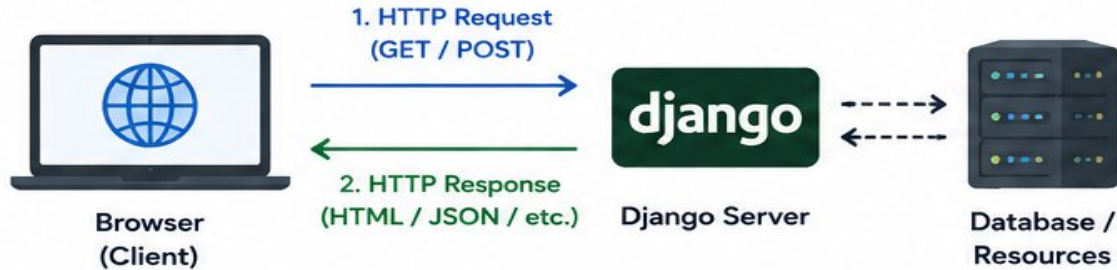
Welcome Aryan

This is the **Response**.

Creating Requests and Responses

In Django, the browser sends a **request** and Django returns a **response**.

How it Works?



Common HTTP Methods

GET	Ask the server to get data. Example: Open a page, view data.
POST	Send data to the server to create or update. Example: Form submission, login.
PUT	Update existing data on the server.
DELETE	Delete data from the server.



Key Points

- Browser sends a request.
- Django (server) processes the request.
- Django returns a response.
- Browser displays the response.

1. Creating a Request (From Browser)

Example 1: GET Request (Open Page)



- Browser sends a GET request.
- Used to open pages or get data.

Example 2: POST Request (Submit Form)

Name:

Email:

- When form is submitted, browser sends a POST request with data.

2. Creating Responses (From Django Server)

Example 1: HTML Response

```
from django.http import HttpResponse

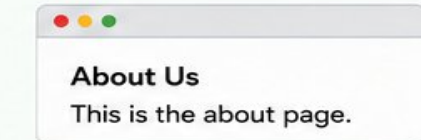
def home(request):
    return HttpResponse("Welcome to Django!")
```



Example 2: Render HTML Template

```
from django.shortcuts import render

def about(request):
    return render(request, "about.html")
```



Example 3: JSON Response

```
from django.http import JsonResponse

def api_data(request):
    data = {"name": "Aryan", "age": 22}
    return JsonResponse(data)
```

```
{ "name": "Aryan", "age": 22 }
```

Complete Flow Example

1. User opens the page / fills a form.
2. Browser sends HTTP Request (GET / POST).
3. Django processes the request.
4. Django returns HTTP Response.
5. Browser displays the response.

Understanding URLs

A URL (Uniform Resource Locator) is the address used to locate and access a resource on a web server.

Django Example

```
path("about/", views.about)
```

If the user visits:

<http://127.0.0.1:8000/about/>

Django sees /about/ and sends the request to:
`views.about`

Understanding URLs

A **URL (Uniform Resource Locator)** is the address used to access a web page or a resource on the internet.

Parts of a URL



How URLs Work in Django



Example URL Patterns

URL	Meaning
/	Home page
/about/	About page
/contact/	Contact page
/student/	Student list page
/student/10/	Details of student with id 10
/student/profile/	Student profile page



Key Points

- URLs help users navigate to different pages.
- In Django, urls.py maps URLs to views.
- Clean and short URLs are better for users and SEO.

Django Code Example

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
    path('contact/', views.contact, name='contact'),
    path('student/', views.student_list, name='student_list'),
    path('student/<int:id>/', views.student_detail, name='student_detail'),
    path('student/profile/', views.student_profile, name='student_profile'),
]
```

views.py

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Home Page")

def about(request):
    return HttpResponse("This is About Page")

def contact(request):
    return HttpResponse("This is Contact Page")

def student_list(request):
    return HttpResponse("This is Student List Page")

def student_detail(request, id):
    return HttpResponse(f"Details of student with id {id}")

def student_profile(request):
    return HttpResponse("This is Student Profile Page")
```

Try in Browser



http://127.0.0.1:8000/about/



This is About Page



http://127.0.0.1:8000/student/10/



Details of student with id 10



http://127.0.0.1:8000/contact/



This is Contact Page

Mapping URLs with Parameters

URL parameters are values we pass through the URL to give information to the View.

For example:

`/student/101/`

Regular expressions in URLs

Regex in URLs lets us define a pattern and control which URLs are allowed to match a particular View.

1. Views.py
2. Urls.py

For eg:

We may want to allow only a URL containing numbers as a Student ID.

/student/101/

/student/abc/

Regular expressions in URLs

Regex allows us to control the pattern of URLs and capture parts of the URL.

Example Objective

We want a URL like `/student/101/`
It should accept only numbers as student id.

urls.py

```
1 from django.urls import re_path
2 from . import views
3
4 urlpatterns = [
5     # Regex: ^start /student/ one or more digits / end$
6     re_path(r'^student/(?P<id>[0-9]+)/$', views.student_detail),
7 ]
```

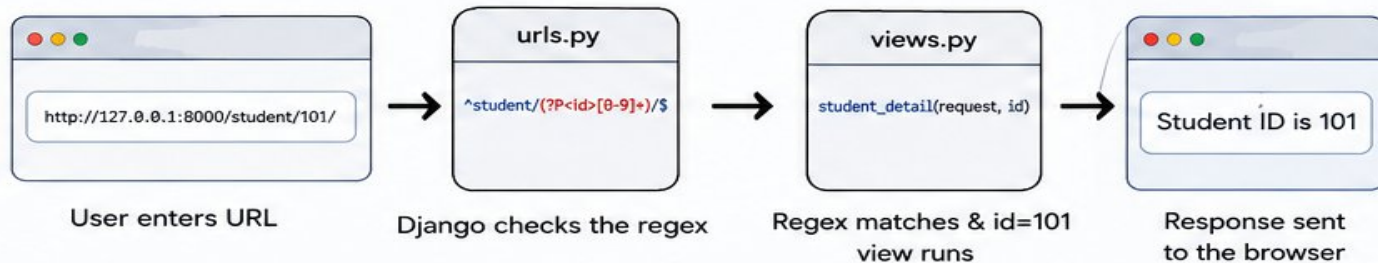
Explanation of the regex

`^` → start of the URL
`student/` → fixed part of the URL
`(?P<id>[0-9]+)` → capture one or more digits and store them in a variable named id
`$` → end of the URL

views.py

```
1 from django.http import HttpResponse
2
3 def student_detail(request, id):
4     return HttpResponse(f"Student ID is {id}")
```

What happens?



Examples

URL	Matches?	Captured id	Result
<code>/student/101/</code>	✓	101	Student ID is 101
<code>/student/25/</code>	✓	25	Student ID is 25
<code>/student/abc/</code>	✗	—	Does not match (only numbers allowed)
<code>/student/12a/</code>	✗	—	Does not match (only numbers allowed)
<code>/student/</code>	✗	—	Does not match (id is required)

Try it Yourself

Open these URLs in your browser

- | | | |
|---|---|---------------------|
| ✓ | <code>http://127.0.0.1:8000/student/101/</code> | → Student ID is 101 |
| ✓ | <code>http://127.0.0.1:8000/student/999/</code> | → Student ID is 999 |
| ✗ | <code>http://127.0.0.1:8000/student/abc/</code> | → Does not match |
| ✗ | <code>http://127.0.0.1:8000/student/12a/</code> | → Does not match |



Key Takeaway:

Regex in URLs helps us accept only the URLs that follow a specific pattern, and also capture parts of the URL as variables.

Error Handling

Error handling is the process of detecting, managing, and responding to errors that occur while processing a user's request.

For example:

- Page doesn't exist → **404 Not Found**
- Server has a problem → **500 Server Error**
- User is not allowed → **403 Forbidden**
- User is not authenticated → **401 Unauthorized**

Error Handling Methods

1. try-except

Used to **catch and handle errors**.

2. get_object_or_404()

Used when getting an object from the database.

3. raise Http404

Used to **manually generate a 404 error**

4. Custom 404 Page

Create your own page when a URL/page is not found.